

# Continual Model-Based Reinforcement Learning with Hypernetworks

Philip Huang

2020-08-18

## Introduction

Effective planning in model-based reinforcement learning (MBRL) and model-predictive control (MPC) relies on the accuracy of the learned dynamics model. In many instances of MBRL and MPC, this model is assumed to be stationary and is periodically re-trained from scratch on state transition experience collected from the beginning of environment interactions. This implies that the time required to train the dynamics model – and the pause required between plan executions – grows linearly with the size of the collected experience. We argue that this is too slow for lifelong robot learning and propose HyperCRL<sup>1</sup>, a method that continually learns the encountered dynamics in a sequence of tasks using task-conditional hypernetworks.

Our method has three main attributes: first, it enables constant-time dynamics learning sessions between planning and only needs to store the most recent fixed-size portion of the state transition experience; second, it uses fixed-capacity hypernetworks to represent non-stationary and task-aware dynamics; third, it outperforms existing continual learning alternatives that rely on fixed-capacity networks, and does competitively with baselines that remember an ever increasing

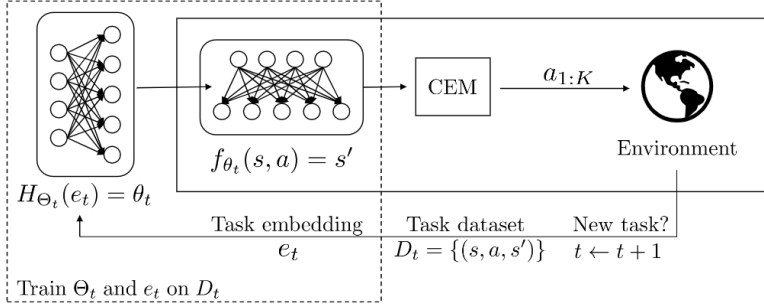
<sup>1</sup> To learn more, you can read our [HyperCRL paper](#) or check out the [code](#).

coreset of past experience. We show that HyperCRL is effective in continual model-based reinforcement learning in robot locomotion and manipulation scenarios, such as tasks involving pushing and door opening.

## Problem Setting

We consider the following problem setting: A robot interacts with the environment to solve a sequence of  $T$  goal-directed tasks, each of which brings about different dynamics while having the same state-space  $S$  and action space  $A$ . The robot is exposed to the tasks sequentially online without revisiting data collected in a previous task.

## Our Method



We have a learned dynamics model, the parameters of which are inferred through a task-conditioned hypernetwork. Given learned task embeddings and parameters of the hypernetwork, we infer parameters of the dynamics neural network. Using this dynamics model, we perform CEM optimization to generate action sequences and execute them in the environment for  $K$  time-steps with MPC. We store the observed transitions in the replay dataset and update the parameters of the hypernetwork and task-embeddings. We repeat this for  $M$  episodes per task, and for each of the  $T$  tasks sequentially.

Figure 1: Door opening with a Panda arm (2x real time)

### Evaluation Tasks

A panda arm must be controlled to open a door. The reward function is designed such that the agent receives higher reward for opening the door to a wider angle. The agent is controlled using an operational space controller (both position and orientation). The different tasks correspond to different shapes of the door knob.

Figure 2: Pushing a non-uniform cube with a Panda arm (2x real time)

A panda arm must be controlled to push a block to a goal location without changing the orientation of the block. The agent is controlled using an operational space controller (only position, orientation of the end-effector is fixed.). The different tasks correspond to different friction coefficients between the cube and the two sides of the table top

Figure 3: Sliding a block towards a goal location (1x real time)

To move the second blue block to its goal pose, the panda arm should first hit block 1, which would slide away and in turn set the second block into sliding motion until stopped by friction. The agent is controlled with an operational space controller similar to the pusher. The different tasks correspond to different friction coefficient between the second block and the table top

### Experiment Results

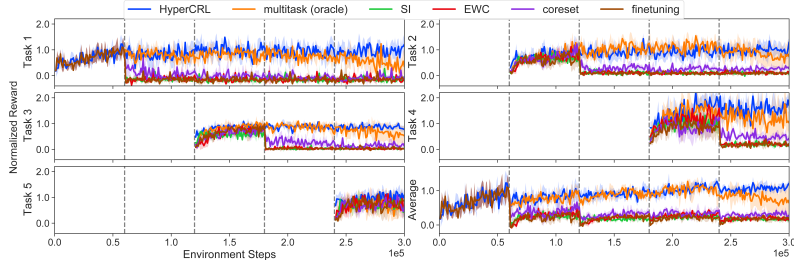


Figure 4: Normalized reward on the Door environment during training

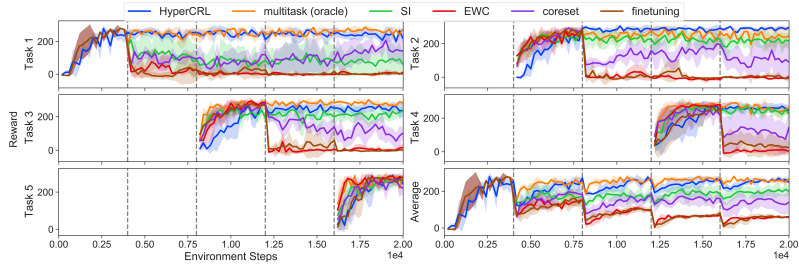


Figure 5: Reward on the Pusher environment during training

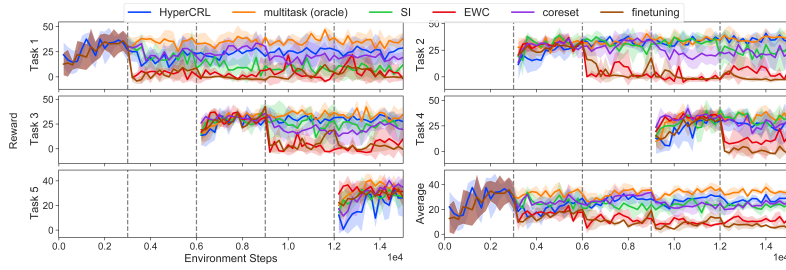


Figure 6: Reward on the Slider environment during training